Smart Office Security and Power Management System Using Face Recognition

Thamer Alotaibi & Abdulaziz Ashy

King Fahd University of Petroleum and Minerals

Computer Engineering Department

COE 454: Internet of Things – Term 242

Abstract—**This report presents an IoT-based smart office system that automates user authentication, door access, and office power control. An ESP32-Camera captures a user image at the entry or exit point and sends it to the server, where face recognition identifies the user and checks active status. For a verified entry, a servo simulates opening the door and an LED simulates powering up the assigned office. For a verified exit, the servo opens the door and the assigned office LED turns off. The prototype demonstrates a low-cost local IoT solution for secure access and power management.**

I. Introduction

Traditional office access methods such as keys, shared passcodes, and manual sign-in sheets can be inconvenient and difficult to monitor. They also do not automatically control office power, so lights or devices may remain on after a user leaves. The motivation for this project is to combine secure access control with simple power management for a smart office environment.

The proposed solution uses an ESP32-Camera at the entry/exit capture point, a server for face recognition and user-state management, and an ESP32 actuator node for the servo and LED outputs. When the server verifies a user at entry, the servo simulates the door opening, the assigned office LED turns on, and the user is marked active. When the same active user is verified at exit, the servo opens again, the assigned office LED turns off, and the user is marked inactive.

The main advantages are contactless authentication, automatic activity logging, local server-side AI processing, and direct linking between access and office power state. The main disadvantages are dependence on Wi-Fi and the running server, sensitivity to face-image quality, and the need to protect the face dataset. Compared with PIN or RFID-only systems, this design is more convenient for users because they can enter hands-free without remembering a code or carrying a card.

II. System Design

The design separates the system into sensing, decision-making, monitoring, and actuation layers. The ESP32-Camera handles image capture, the server performs the heavier AI recognition and active-state logic, and the actuator ESP32 performs the physical simulation. A servo is used to simulate the office door, while an LED is used to simulate powering the assigned office up or down. MQTT was selected because it is lightweight and suitable for event-based IoT communication.

A. System Architecture

The system has four main parts: the ESP32-Camera node, the server, the web dashboard, and the ESP32 actuator node. The camera node captures a face image and sends it to the server. The server identifies the user, checks active status, updates login/logout state, and publishes the actuator command. The dashboard displays active users, login history and latest attempt. The actuator node controls the servo door simulation and office-power LED.

- ESP32-Camera captures face images at the entry/exit point and publishes image data.
- Server performs face recognition, verifies users, and manages active/inactive state.
- Web dashboard shows active users, login/logout history, latest attempt, and server health.
- ESP32 actuator node receives commands, drives the servo door simulation, and switches the assigned office LED power indicator. Fig. 1 illustrates the entry and exit workflow, showing how a captured face image leads to authentication, servo door control, and assigned office LED power control.

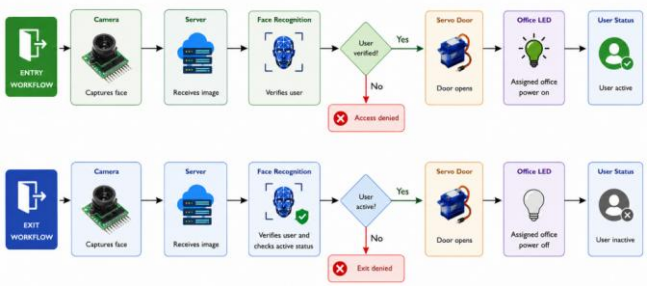


Fig. 1. Entry and exit workflow for secure access and office power management.

B. Network Architecture

All devices are intended to run on the same Wi-Fi local area network. The MQTT broker and dashboard run on the server, while the ESP32 nodes connect to the server IP address. Keeping the system local reduces delay and avoids sending face images to a cloud service.

The main MQTT topics are faceauth/esp32_1/image for image uploads, faceauth/esp32_1/reject for rejected users, and faceauth/esp32_2/door for actuator commands. The payload open,<person_id>,login represents verified entry: the servo

opens and the assigned office LED turns on. The payload open,<person_id>,logout represents verified exit: the servo opens and the LED turns off. The dashboard is served locally on port 5055.

C. Computing Architecture

The computing architecture is an edge-server model. The ESP32-Camera node performs sensing and publishes image bytes. The server runs Python, receives MQTT messages, performs face matching using InsightFace/ArcFace model, manages active-user state, and stores history. The actuator ESP32 runs MicroPython and performs real-time servo and LED control.

The face database is stored in face_database.npy and can be rebuilt from the people folder. Login and logout history is stored in history.json. The server shares dashboard state safely because the MQTT listener and Flask dashboard run at the same time.

D. Technologies and Protocols

The following technologies and protocols were used in the prototype:

- Python: server application, face recognition wrapper, dashboard backend, and local data handling.
- MicroPython: ESP32-Camera node and ESP32 actuator node firmware. The ESP32-Camera implementation uses the OV2640 MicroPython camera library [1].
- MQTT: lightweight publish/subscribe communication between ESP32 nodes and the server.
- InsightFace, ArcFace, OpenCV, NumPy, and ONNX Runtime: face detection, embedding extraction, and matching.
- Flask, HTML, CSS, and JavaScript: live dashboard and JSON API.
- GPIO/PWM control: servo door simulation and LED-based office power simulation.

As shown in Table I, the implementation is organized into server, AI, dashboard, camera, actuator, and data files, so each project file has a clear role in the complete smart office workflow.

TABLE I
PROJECT FILES AND PURPOSES

File or folder	**Purpose**
main_dashboard.py	Server app: receives images, authenticates users, updates state/history, and sends actuator commands.
ai_model.py	Face recognition wrapper and face database builder.
dashboard.py / templates	Dashboard UI/API for activity, latest attempt, health, and server status.
ESP32_1_Camera.py	Camera node: captures entry/exit face images and publishes them to MQTT.
ESP32_2_ Servo_LED.py	Actuator node: controls the servo door simulation and office-power LED.
people/ and data files	Registered face images, embeddings, history, and latest-attempt image.

III. Performance Evaluation

The network performance was evaluated for one authenticated access process using MQTT QoS 0 over TCP/Wi-Fi. The evaluated path is ESP32-CAM -> server -> ESP32 actuator node. This path is divided into two network processes. The first process is ESP32-CAM -> server, where the ESP32-CAM sends the JPG image to the server. The second process is server -> ESP32 actuator node, where the server sends the authentication command to the actuator ESP32 that controls the servo door and the assigned office LED. Server processing latency is ignored in this network evaluation, so the calculation focuses only on network transmission overhead and latency.

The Wi-Fi latency was measured by pinging the ESP32 100 times using a 1-byte ping payload. The measured round-trip time had a minimum of 11 ms, a maximum of 103 ms, and an average RTT of 35 ms. A standard 1500-byte MTU was assumed for frame-level overhead. Therefore, the ESP32-CAM -> server image publish is carried over 13 frames, while the server -> ESP32 actuator command publish is carried over one frame. Table II presents the payload, layer overhead, latency, throughput, and goodput results for both network processes.

TABLE II
NETWORK PERFORMANCE EVALUATION RESULTS

Metric	ESP32-CAM -> Server	Server -> ESP32 Actuator
Payload	17,608 B JPG image	13 B command: open,01,login
Number of frames	13 frames	1 frame
Application overhead	28 B MQTT QoS 0 PUBLISH	25 B MQTT QoS 0 PUBLISH
Transport overhead	13 x 20 B = 260 B TCP headers	1 x 20 B = 20 B TCP header
Network overhead	13 x 20 B = 260 B IPv4 headers	1 x 20 B = 20 B IPv4 header
Datalink overhead	13 x 34 B = 442 B Wi-Fi headers	1 x 34 B = 34 B Wi-Fi header
Total overhead	28 + 260 + 260 + 442 = 990 B = 0.990 KB	25 + 20 + 20 + 34 = 99 B = 0.099 KB
Total transmitted data	17,608 B + 990 B = 18,598 B = 18.598 KB	13 B + 99 B = 112 B = 0.112 KB
Protocol overhead	(990 / 18,598) x 100 = 5.32%	(99 / 112) x 100 = 88.39%
Wi-Fi latency	Average RTT = 35 ms per frame	
Process latency	13 x 35 ms = 455 ms = 0.455 s	1 x 35 ms = 35 ms = 0.035 s
Throughput	148,784 bits / 0.455 s = 327,000 bps = 327.00 kbps	896 bits / 0.035 s = 25,600 bps = 25.60 kbps
Goodput	(17,608 B x 8) / 0.455 s = 309,596 bps = 309.60 kbps	(13 B x 8) / 0.035 s = 2,971 bps = 2.97 kbps
Overall network latency	455 ms + 35 ms = 490 ms = 0.49 s	

According to Table II, the overall network latency is 490 ms, with 455 ms from ESP32-CAM to server and 35 ms from server to ESP32 actuator. Most of the delay comes from image transmission because the JPG image requires 13 frames, while the actuator command requires only one frame. The ESP32-CAM to server path achieved 327.00 kbps throughput and 309.60 kbps goodput, showing a small difference because the 0.990 KB header overhead is low compared with the 17.608 KB image payload. In contrast, the server to actuator path achieved 25.60 kbps throughput and only 2.97 kbps goodput because the 0.099 KB header overhead is much larger than the 0.013 KB command payload. Therefore, header overhead has a limited effect on large image messages but a major effect on very small command messages.

For the computing evaluation, the workload is divided according to device capability. The ESP32-CAM performs the light task of capturing the JPG image and publishing it to MQTT. The actuator ESP32 performs another light task by parsing the short command, such as open,01,login, and updating the servo and LED outputs.

The heavier computing task is kept on the server. The server receives the image, runs face detection and matching, checks the user active/inactive state, updates the dashboard data, and publishes the actuator command. This server-side computing choice avoids running the AI model on the ESP32 nodes and keeps the embedded devices focused on sensing and actuation.

The AI performance was also evaluated from a computing perspective. The face-recognition workload is the most computationally expensive part of the system because it requires receiving the JPG image, detecting the face, extracting the face embedding, and comparing it with the registered users in the face database. This task is executed on the server instead of the ESP32 nodes because the server has higher processing power and memory capacity.

In the prototype, the AI model only compares the captured face against a small number of registered users, so the matching step remains lightweight and suitable for real-time access control. However, the observed face-recognition confidence was less than 65%, mainly due to the poor image quality produced by the camera. Low camera quality can reduce face clarity, lighting consistency, and facial feature detail, which affects the confidence score of the recognition model. As the number of registered users increases, the AI processing time may also increase because more embeddings must be compared. However, keeping the AI workload on the server makes the system easier to scale than running face recognition directly on the ESP32-CAM.

IV. Problem Faced

Several implementation problems were addressed during integration and testing:

- Large camera images: high-resolution JPEG images can exceed ESP32 memory or MQTT payload limits. The camera code includes MAX_IMAGE_BYTES and notes to lower the resolution when needed.
- Face-recognition sensitivity: lighting, pose, and image quality affect confidence. Multiple images per user were stored in the people folder and an authentication threshold was used.

No core prototype task is intentionally left incomplete. The remaining limitations are mainly improvements: MQTT authentication/TLS, a stronger database instead of JSON files, a real relay-controlled office power circuit instead of an LED, a physical door-position sensor, an enrolment interface, and more testing under different lighting conditions.

V. Team Structure

The work was divided so that both members contributed to hardware, software, testing, and documentation.

TABLE III
TEAM RESPONSIBILITIES

Team member	Main responsibilities
Thamer Alotaibi	ESP32-CAM, Data preparation, AI model implementation, MQTT server setup.
Abdulaziz Ashy	Actuator ESP32, servo door simulation, office-power LED behavior, MQTT command testing, Dashboard implementation.
Both members	Full workflow integration, registered-user login/logout testing, rejected-attempt testing, final verification, report writing.

VI. Conclusions

This project implemented a smart office security and power management prototype using an ESP32-Camera, MQTT, server-side AI face recognition, a Flask dashboard, and an ESP32 actuator node. The servo simulates door access, the LED simulates assigned office power, and the server manages whether each recognized user is active or inactive.

The main achievement is the integration of sensing, local AI processing, user-state management, monitoring, and actuation into one IoT workflow. Future improvements can include encrypted MQTT, a database backend, real office-power relay control, separate entry and exit camera deployment, improved camera calibration, and larger-scale testing with more users.

VII. References

[1] Nmato, "micropython-ov2640: MicroPython class for OV2640 Camera," GitHub. Available: https://github.com/namato/micropython-ov2640